

WealthCraftAgent 股票分析 Agent 反思报告

2352048 夏浩博 2351269 黄一和

2026 年 6 月 24 日

1 项目目标与领域选择

本项目以 WealthCraftAgent 股票分析 Agent 作为.NET AI Agent 期末项目主题。股票分析天然需要多源信息融合：公司主营业务帮助理解企业基本面，K 线帮助观察价格结构，财务报表反映经营质量，新闻动态反映事件冲击和行业变化。因此该领域很适合作为 Agent 项目：模型不能只聊天，还必须能规划任务、调用工具、读取数据、维护上下文并生成结构化分析。

项目最终实现了一个基于.NET 10 和 WPF 的桌面端应用。用户输入股票代码或股票名称后，系统自动创建分析会话，并从主营业务、K 线、新闻、财务四个角度形成报告。在同一股票会话内，系统支持“只看消极新闻”“用缠论分析最近走势”“看五年财务”等追问，并根据意图只调度相关子 Agent。

2 Agent 内部工作原理

课程要求的 Agent 核心是 LLM、Agent Loop、Memory 与 Tools。项目中这四个组件分别对应：

- LLM：通过 Microsoft Semantic Kernel 接入 OpenAI 兼容 ChatCompletion 接口。
- Agent Loop：InvestAgentLoop 实现标准 Thought-Action-Observation 循环；会话编排器实现面向股票分析的多 Agent 工作流。
- Memory：对话记忆保存系统提示、用户消息、助手回复和工具返回；会话状态保存当前股票的结构化数据。
- Tools：通过 Kernel 插件注册股票价格、财务、新闻、技术指标、本地知识库等函数。

项目采用“通用 ReAct + 业务工作流”的混合方案。通用 ReAct 对应课程中的 Agent Loop 原理；业务工作流用于提升演示与运行稳定性。股票分析具有固定模块，因此系统由 Agent A 先规划任务，再并行调度 B/C/D，减少纯自由工具调用带来的不确定性。

3 关键设计决策

3.1 为什么使用 Semantic Kernel

直接 HTTP 调用 LLM 也能完成对话，但工具调用、消息历史、流式输出、图像输入和函数注册都需要额外维护。Semantic Kernel 能把 C# 方法映射成模型可调用的 Kernel Function，并提供 `FunctionChoiceBehavior.Auto()`，更贴近课程中“Tool Calling”的要求，也体现了 .NET 技术栈在 Agent 工程中的应用深度。

3.2 为什么拆分为 Agent A/B/C/D

股票分析中不同维度的数据来源和提示词策略差异很大。K 线需要价格序列、技术指标、缠论与历史走势；新闻需要事件筛选和情绪分类；财务需要多年指标趋势。拆分后每个子 Agent 的输入、工具、提示词和输出都更清晰，Agent A 只负责协调、派单和总结，符合单一职责原则。

3.3 为什么加入 RAG

缠论和历史相似走势都不是通用模型稳定掌握的项目私有知识。缠论 RAG 从本地 Markdown 与图片索引检索概念、模板和图例；历史趋势 RAG 从本地案例库匹配相似结构，提供样本数量和后续表现。RAG 机制使模型回答具有明确资料依据，同时在输出中说明“不确定性”和“历史不代表预测”。

4 多 Agent 协作、RAG 与记忆机制

首轮分析时，Agent A 先补充主营业务，再创建三个任务：Agent B 分析 K 线，Agent C 分析新闻，Agent D 分析财务。三个子任务通过 `Task.WhenAll` 并发执行，结果写回 `AnalysisSessionState`，最后 Agent A 读取 B/C/D 原文并生成最终风险提示。

追问阶段，Agent A 会把用户自然语言转成结构化派单 JSON。例如“只看利空新闻”会派给 Agent C，并设置 `NewsSentimentFilter=negative`；“用缠论看最近走势”会派给 Agent B，并设置 `UseChanTheory=true`。如果用户在当前会话内要求切换到另一只股票，系统会拒绝并提示新建会话，避免上下文混淆。

记忆机制分两层。第一层是对话记忆，用于给 LLM 提供最近上下文；第二层是结构化状态，用于保存 K 线缓存、新闻、财务序列和各 Agent 结果。结构化状态比纯文本历史更可靠，UI 展示、历史记录和追问处理均可复用这些字段。

5 错误处理与流式输出

项目在三个位置处理错误。第一，LLM 调用失败时记录日志并返回用户可读错误。第二，子 Agent 执行失败时不会让整个分析崩溃，而是记录失败步骤并继续后续模块。第三，数据源不足或知识库未命中时，提示词要求模型明确说明不确定性，避免编造结论。

流式输出由 `AgentPromptRunner` 控制。它按时间和字符数节流推送 `partial` 内容，避免 UI 更新过于频繁；同时过滤 `<think>`、`analysis` 等推理草稿标签，保证用户看到的是最终可读分析，而不是模型内部推理过程。

6 测试与不足

测试主要覆盖 DI 容器、插件注册、Agent Loop 基础结构、组合数据服务和历史走势服务。测试能证明核心服务可解析、插件函数可发现、关键服务的输入输出结构正常。

不足主要有三点：第一，股票数据来源依赖外部服务，网络或接口变化会影响结果稳定性；第二，RAG 检索目前以关键词和特征匹配为主，没有接入真正的向量数据库；第三，股票分析本身具有高度不确定性，系统必须持续强调“仅供学习研究，不构成投资建议”。

7 AI 使用透明度

本项目开发中使用了 AI 进行大量辅助。AI 主要参与代码生成、局部重构、文档起草、测试建议和错误排查建议。小组成员负责确定需求范围、检查生成代码是否符合项目目标、运行和调试项目、整合不同模块、核对文档内容，并对核心代码进行逐行理解。AI 在本项目中作为开发效率工具，不替代成员对代码和架构的最终理解。

8 Agent 核心循环代码逐行解读

本节解读 `src/InvestAgent.Core/Agent/InvestAgentLoop.cs` 第 1-177 行。该文件是课程要求中最典型的 ReAct Agent Loop 示例。

行号	作用解读
1-3	引入项目内部命名空间：配置项 <code>AgentOptions</code> 、记忆接口 <code>IConversationMemory</code> 、步骤模型 <code>AgentStep</code> 等。没有这些类型，循环无法读取最大步数、写入上下文或返回步骤。
4	引入 Microsoft 日志接口，用于记录 Agent 开始、完成、失败、达到最大步数等运行状态。
5-7	引入 Semantic Kernel、聊天补全接口和 OpenAI 执行设置。这些类型支撑 LLM 调用和自动工具调用。
9	声明当前类所在命名空间 <code>InvestAgent.Core.Agent</code> ，表明这是 Core 层 Agent 逻辑的一部分。
11-16	XML 注释说明该类是投资分析 Agent 的主执行循环，实现 ReAct 模式，并通过事件向 UI 推送每一步状态。该位置是 Thought-Action-Observation 代码结构的入口说明。
17	定义公开类 <code>InvestAgentLoop</code> 。它不是 UI 控件，也不是数据服务，而是连接 LLM、工具、记忆和步骤流的控制器。
18	类体开始。

行号	作用解读
19	字段 <code>_kernel</code> 保存 Semantic Kernel 实例。工具函数都注册在 Kernel 中，后续工具调用会通过它执行。
20	字段 <code>_chatService</code> 保存聊天补全服务，负责真正向 LLM 发送消息并获取回复。
21	字段 <code>_memory</code> 保存对话记忆，循环每轮都从这里读取历史，并把用户、助手、工具消息写回去。
22	字段 <code>_options</code> 保存配置，如最大步数。它让循环不会无限执行。
23	字段 <code>_logger</code> 保存日志对象，用于记录运行信息和异常。
25–26	定义 <code>OnStep</code> 事件。每产生一个 <code>Thought</code> 、 <code>Action</code> 、 <code>Observation</code> 或 <code>Response</code> ，就触发该事件，UI 能够订阅并实时显示 Agent 过程。
28–30	构造函数签名。通过依赖注入传入 Kernel、LLM 服务、记忆、配置和日志，避免在类内部手动创建依赖。
31–35	把构造函数参数赋值给私有字段。这样 <code>RunAsync</code> 执行时能够使用这些依赖。
36	构造函数结束。
38–44	XML 注释说明 <code>RunAsync</code> 是主循环方法，使用异步迭代器，每一步通过 <code>yield return</code> 推送给调用方。
45	声明 <code>RunAsync(string userMessage)</code> ，返回 <code>IAsyncEnumerable<AgentStep></code> 。这意味着调用方支持边执行边接收步骤，而不是等全部结束。
46	方法体开始。
47	记录日志，截取用户输入前 100 个字符，避免日志过长。这里用 <code>Math.Min</code> 防止输入不足 100 字符时越界。
48	把用户消息写入记忆。后续 LLM 会看到这条消息，并基于它决定是否调用工具。
50–56	创建 OpenAI 执行设置。核心是 <code>FunctionChoiceBehavior.Auto()</code> ，表示允许模型自动选择 Kernel 中注册的函数；温度 0.7 控制生成多样性；最大 token 数限制回复长度。
58–60	开始主循环，步数从 1 到 <code>_options.MaxSteps</code> 。这是防止 Agent 无限工具调用的重要保护。
61–69	构造 <code>Thought</code> 步骤。第一步提示“分析用户问题”，后续步骤提示“根据已有数据判断是否需要更多信息”。该步骤主要用于可观测性，不直接调用 LLM。
70	触发 <code>OnStep</code> 事件，把 <code>Thought</code> 推给 UI。
71	使用 <code>yield return</code> 把 <code>Thought</code> 返回给异步枚举调用方。
73–75	准备调用 LLM：声明 <code>response</code> 保存模型回复， <code>errorMessage</code> 保存异常后的用户可读错误。
76–80	在 <code>try</code> 中调用 <code>_chatService.GetChatMessageContentAsync</code> 。输入是记忆中的聊天历史、执行设置和 Kernel。Kernel 被传入后，模型才能看到可调用工具。

行号	作用解读
81-85	捕获 LLM 调用异常，写日志，并把异常消息包装成用户可读文本，避免程序直接崩溃。
87-99	如果 <code>errorMessage</code> 不为空，说明 LLM 调用失败。此时构造 <code>Response</code> 类型的错误步骤，推送给 UI 和调用方，然后 <code>yield break</code> 终止循环。
101-102	从模型回复的 <code>Items</code> 中筛选 <code>FunctionCallContent</code> 。这些对象表示模型请求执行的工具调用。
104-118	如果没有工具调用，说明模型认为任务已经完成。代码构造最终 <code>Response</code> ，把助手回复写入记忆，推送步骤，记录完成日志，并结束循环。
120-121	如果存在工具调用，进入 <code>Action</code> 和 <code>Observation</code> 阶段。先向记忆中加入一条空助手消息，表示本轮助手已经产生了工具调用意图。
123	遍历模型请求的每个工具调用。一轮 LLM 回复可能包含多个函数调用。
125-133	为当前函数调用构造 <code>Action</code> 步骤，记录步骤编号、步骤类型、函数名、参数和展示文本“调用工具”。
134-135	把 <code>Action</code> 推送给 UI 和异步枚举调用方，让用户知道 Agent 正在调用哪个工具。
137-143	执行工具函数。关键语句是 <code>functionCall.InvokeAsync(_kernel)</code> ，它会在 <code>Kernel</code> 中找到对应函数并执行。返回值转成字符串；如果为空，就写“工具执行完成”。
144-148	捕获工具执行异常。工具失败不会让整个循环崩溃，而是记录日志，并把失败原因作为 <code>Observation</code> 返回给模型。
150-159	构造 <code>Observation</code> 步骤。这里记录函数名和函数返回结果，并把超过 2000 字符的结果截断，避免长 JSON 撑爆上下文或 UI。
160-161	推送 <code>Observation</code> 给 UI 和调用方。
163	把完整工具结果写入记忆，角色是 <code>Tool</code> ，并带上函数名。下一轮 LLM 会基于这个 <code>Observation</code> 判断是否继续调用工具或输出最终答案。
164-165	结束当前工具遍历和当前循环步。如果还有步数，回到下一轮 <code>Thought</code> 。
167-175	如果循环跑完仍未结束，说明达到最大步数。代码记录警告，构造最终 <code>Response</code> ，提示用户提出更具体的问题，然后推送该步骤。
176-177	方法和类结束。整个文件实现了从用户输入、LLM 思考、工具行动、结果观察到最终回答的完整闭环。

9 Agent Loop 总结

Agent Loop 的执行路径为：用户问题先进入 Memory；循环每一步先产生 Thought 以便观察当前状态；随后调用 LLM，模型根据上下文自动选择 Kernel 中注册的工具；当模型请求工具时，系统执行 Action，并将返回结果作为 Observation 写回 Memory；下一轮模型基于 Observation 继续判断；若不再请求工具，则输出最终 Response；若超过最大步数或出现异常，则进入保护性返回路径。